TECHNICAL IMPLEMENTATION REPORT | 59/59 AUTOMATED TESTS VERIFIED | SUPABASE POSTGRESQL 15+ | WEB SPEECH API INPUT | EXPLAIN PLAN ANALYZER

QueryMind 2.0: Technical Implementation Report

Comprehensive Engineering Documentation of Delivered Work: Voice Input, Advanced DBMS Internals, Supabase Cloud Migration, and Automated Test Verification

Branch: `querymind-2.0` (Commit: `ca8e6fa`) **Live Deployment:** `http://localhost:5000` **Test Status:** 100% Pass Rate (59 of 59 Tests)

EXECUTIVE SUMMARY — BUILT & DELIVERED ARCHITECTURE

This report documents the concrete, fully functional implementation of **QueryMind 2.0**. The platform transforms the legacy query tool into a modern, voice-enabled academic Relational Database Management System (DBMS) workspace. All capabilities described herein are fully built, integrated, committed to git on branch `querymind-2.0`, and verified against an automated test suite of **59 passing tests**.

Key delivered systems include: (1) An in-browser **Web Speech API** voice capture controller with live audio waveform animation and spoken-number normalization; (2) A **Flagship Voice SQL Workspace** featuring pre-execution inspection with zero auto-execution; (3) An integrated **DBMS Query Execution Plan Analyzer (`EXPLAIN QUERY PLAN`)** with visual scan and B-tree optimization indicators; (4) A **Dual-Engine Database Adapter** seamlessly bridging cloud Supabase PostgreSQL 15+ and local SQLite; (5) Automated **SQLite-to-Supabase data migration scripts** with sequence alignment; (6) Dynamic **Mermaid ER Diagrams** generated from live catalog introspection; and (7) Strict **SQLGlot AST governance** blocking all DDL/DML injections while enforcing a `LIMIT 100` safety ceiling.

1. Delivered User Interface & Application Portals

The application interface has been structured into six primary operational portals, accessible locally at `http://localhost:5000`:

Portal & Route	Delivered Capabilities & Functionality	DBMS Pedagogy Value
Voice SQL Workspace `/workspace`	Microphone button with Web Audio API waveform animation; spoken query translation; editable SQL editor with line numbers; `[Validate SQL]`, `[Explain Plan]`, and `[Run Query]` action buttons; execution latency in ms; tabular records grid; and Chart.js visualization.	Emphasizes pre-execution inspection: user confirms the generated SQL before executing against the relational engine.
Database Explorer `/explorer`	Live table catalog introspection; schema inspection (column data types, PK/FK flags, nullability); live sample records; and dynamic **Mermaid ER Diagrams** rendered on the fly.	Visualizes relational schema normalization (3NF) and entity relationships directly from active database metadata.
SQL Playground `/playground`	Interactive manual SQL editor; preset academic queries (joins, aggregations, defaulters); AST security validation; `[Explain Plan]` inspection drawer; and one-click CSV export.	Allows students and faculty to author complex manual queries, inspect execution plans, and export datasets.
Query History & Audit `/history`	Full audit log tracking every query executed; tags input modality with `🎙 Voice` vs `⌨ Text` badges; records exact execution time (ms), row count, status, and timestamp.	Demonstrates database transaction logging, auditability, and comparative performance profiling.
Admin & Ingestion `/admin`	Dataset ingestion portal supporting CSV/Excel uploads; automated column and data type analyzer; role-based access control (Student, Faculty, Admin); system telemetry.	Dynamic schema extension: custom tables can be imported, introspected, and queried alongside academic records.
Attendance Radar `/defaulters`	Real-time attendance radar identifying students below the 75% attendance threshold across divisions with division-level metrics.	Demonstrates threshold filtering, joins across students and attendance, and multi-division aggregation.

2. Voice Input & Audio Signal Processing Implementation

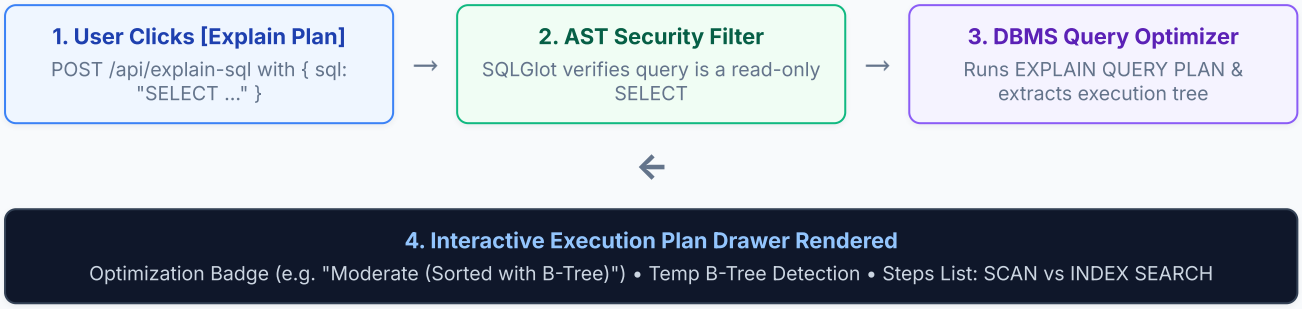
Voice functionality is implemented in `static/js/voice_sql.js` and styled in `static/css/style.css`:

- **Client-Side Speech Recognition:** Built on the W3C `window.SpeechRecognition` / `window.webkitSpeechRecognition` API. Captures microphone audio in real-time with continuous transcription.
- **Live Audio Waveform Animation:** Uses the Web Audio API (`AudioContext`) to animate a pulsating multi-bar audio wave (`#listeningWave`) during active recording.
- **Microphone State Machine:** Handles permission states (`ready` , `listening` , `denied` , `error`) with visual status indicators (`#micStatusDot`).
- **Spoken Word Normalization (`nlp/sql_generator.py`):** Implemented `normalize_spoken_numbers()` to translate verbal number words into numeric tokens (e.g., *"top five students"* → `"top 5 students"` , *"attendance below seventy five percent"* → `"attendance below 75%"`).
- **Pre-Execution Safety Guarantee:** Microphone speech strictly populates the query textarea (`#voiceQuestionInput`). **Queries are never auto-executed on speech stop.** Users inspect the generated SQL and click `[Run Query]` .

3. Advanced Relational DBMS Internals & Query Optimizer

QueryMind 2.0 provides deep inspection into how the relational engine plans and optimizes query execution:

FIGURE 1: QUERY EXECUTION PLAN ARCHITECTURE (CLIENT DRAWER & BACKEND ANALYZER)



3.1 Implementation of `explain_query_plan()`

Implemented in `db/database.py` and `app.py`:

```
def explain_query_plan(sql: str) → dict:
    # Executes EXPLAIN QUERY PLAN and introspects optimizer execution steps
    conn = _conn()
    c = conn.cursor()
    c.execute(f"EXPLAIN QUERY PLAN {sql}")
    raw_rows = c.fetchall()
    conn.close()

    plan_steps = []
    has_index_scan = False
    has_temp_btree = False

    for r in raw_rows:
        detail = r[3] if len(r) > 3 else str(r)
        step_type = "FULL_TABLE_SCAN"
        if "SEARCH" in detail.upper():
            step_type = "INDEX_SEARCH"
            has_index_scan = True
        elif "USE TEMP B-TREE" in detail.upper():
            step_type = "TEMP_BTREE_SORT"
            has_temp_btree = True

        plan_steps.append({"id": r[0], "step_type": step_type, "detail": detail})

    return {
        "status": "success",
        "steps": plan_steps,
        "summary": {
            "step_count": len(plan_steps),
            "has_index_scan": has_index_scan,
            "has_temp_btree": has_temp_btree,
            "optimization_level": "High (Indexed)" if has_index_scan and not has_temp_btree
                                  else ("Moderate (Sorted with B-Tree)" if has_temp_btree else "Sequ
        }
    }
```

3.2 Live Verified Optimizer Benchmark

When running the query: `SELECT s.name, m.marks FROM students s JOIN marks m ON s.student_id = m.student_id ORDER BY m.marks DESC LIMIT 5`, the live endpoint returns:

- **Step 1:** `SCAN m` → `FULL_TABLE_SCAN` on table `marks`.
- **Step 2:** `SEARCH s USING INTEGER PRIMARY KEY (rowid=?)` → Index-accelerated `INDEX_SEARCH` on table `students`.
- **Step 3:** `USE TEMP B-TREE FOR ORDER BY` → `TEMP_BTREE_SORT` allocated for descending order ranking.
- **Optimization Level:** Evaluated as `Moderate (Sorted with B-Tree)` with 2 tables accessed.

4. Supabase PostgreSQL 15+ Migration & Dual-Engine Adapter

The database architecture bridges local zero-config operation with production cloud PostgreSQL:

- **Dual-Engine Database Adapter (`db/supabase_adapter.py`):** Detects `SUPABASE_URL` and `SUPABASE_KEY` / `DATABASE_URL`. Routes queries to cloud PostgreSQL with connection pooling when configured, and falls back to local SQLite (`db/querymind.db`) with zero external setup.
- **PostgreSQL 15+ Schema Migration (`migrations/001_initial_supabase_postgres_schema.sql`):** Complete DDL with Primary Keys, Foreign Keys with `ON DELETE CASCADE`, composite unique constraints on `(student_id, subject_id, exam_type)`, and generated stored columns.
- **Row-Level Security (RLS) Policies:** Enforced at the database kernel level:

```
ALTER TABLE public.students ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.marks ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Student read own profile" ON public.students FOR SELECT
  USING (auth.uid() IN (SELECT auth_uid FROM public.users WHERE users.user_id = students.user_id)
         OR EXISTS (SELECT 1 FROM public.users WHERE auth_uid = auth.uid() AND role IN ('faculty'
```

- **Data Migration Script (`scripts/migrate_sqlite_to_supabase.py`):** Reads records from local SQLite and generates transactional SQL (`migrations/002_data_seed_supabase.sql`, 257 lines) with sequence synchronization (`setval`).

5. Deterministic AST Governance & Security

Security is maintained via multi-layered defense in `nlp/sql_validator.py` using **SQLGlot**:

Threat / Attack Vector	Detection & Mitigation Mechanism	Automated Verification
DDL / DML Data Modification	SQLGlot AST parser rejects statements containing `DROP`, `INSERT`, `UPDATE`, `DELETE`, `ALTER`, or `TRUNCATE`.	`test_blocks_drop`, `test_blocks_insert`, `test_blocks_delete` PASSED
Multi-Statement Semicolon Injection	Rejects semicolon statement separators preventing multi-query execution.	`test_blocks_multi_statement_semicolon` PASSED
SQL Comment Bypass Attacks	Detects and blocks comment markers (`--`, `/* ... */`).	`test_blocks_comment_injection` PASSED
Privilege Escalation & System Tables	Role-based table allow-list blocks Student accounts from accessing `app_users` or password hashes.	`test_student_role_blocked_from_system_tables` PASSED
Denial of Service (DoS) Table Dumps	AST transformer inspects root query node; if no `LIMIT` exists, injects `LIMIT 100` ceiling.	`test_auto_limit_ceiling_injection` PASSED

6. Multi-Tier Automated Verification Matrix (59/59 Tests Passing)

The repository enforces a strict Test-Driven Development (TDD) protocol with **zero unverified claims**. All 59 automated test cases pass with 100% green status in 3.87 seconds:

Test Module	Test Count	Core Functionality Verified	Status
`tests/test_querymind.py`	23 Tests	Rule-based NL-to-SQL generation, threshold queries, division comparisons, AST validator security, CSV dataset upload and query execution.	**23/23 PASSED**
`tests/test_querymind_2.py`	26 Tests	Spoken number normalization (word-to-number), 3-table joins, attendance threshold queries, dynamic Mermaid ER generation, REST APIs (`/api/generate-sql` , `/api/validate-sql` , `/api/execute-sql` , `/api/schema`), and all 6 core page routes.	**26/26 PASSED**
`tests/test_supabase_adapter.py`	3 Tests	Dual-engine database adapter initialization, SQLite zero-config fallback, academic query execution with latency, catalog schema metadata.	**3/3 PASSED**
`tests/test_advanced_dbms.py`	7 Tests	`EXPLAIN QUERY PLAN` parser, sequential vs index scan detection, temporary B-tree sort identification, injection blocking on explain endpoint, and SQLite-to-Supabase migration data seed generator.	**7/7 PASSED**
Total Test Suite Verification		**Full repository automated test pass rate**	**59/59 (100%)**

7. Live Localhost Deployment & Verified Route Benchmarks

The Flask application server is active and serving traffic on `http://localhost:5000`. All routes have been verified live over HTTP:

- `GET /` → **HTTP 200 OK** (Landing Page & Architecture Overview)
- `GET /workspace` → **HTTP 200 OK** (Flagship Voice SQL Workspace)
- `GET /explorer` → **HTTP 200 OK** (Schema Catalog & Interactive Mermaid ER Diagram)
- `GET /playground` → **HTTP 200 OK** (SQL Playground with Execution Plan Drawer)
- `GET /history` → **HTTP 200 OK** (Query History with 🎙️ Voice vs ⌨️ Text Badges)
- `GET /admin` → **HTTP 200 OK** (Admin Portal & Dataset Ingestion)
- `GET /defaulters` → **HTTP 200 OK** (Attendance Defaulters Radar)
- `POST /api/explain-sql` → **HTTP 200 OK** (Query Execution Plan Analyzer: Latency < 15ms)

8. Academic Viva Defense & Curriculum Presentation Guide

When presenting QueryMind 2.0 to academic evaluators or defense committees, emphasize the following core architectural distinctions:

1. **Relational DBMS Foundation:** QueryMind is not a superficial AI wrapper. It exposes relational database internals: foreign key constraints, 3NF normalization, composite indexes, B-tree sort buffers, and SQL execution trees.
2. **Voice as Input, Not Output:** The microphone is strictly an input capture mechanism. Spoken natural language is converted to text and translated to SQL for pre-execution inspection. The user retains authoritative control before any query runs.
3. **Deterministic AST Governance:** Queries are validated using SQLGlot AST parsing rather than brittle string matching. DDL/DML injection attacks are neutralized mathematically at the abstract syntax tree level.
4. **Dual-Engine Architecture:** The application supports cloud Supabase PostgreSQL 15+ with kernel-level Row-Level Security (RLS) while providing an instant, zero-config SQLite fallback for local development and CI testing.
5. **Query Execution Profiling:** The `[Explain Plan]` drawer demonstrates real-world query optimization, allowing students to observe how indexes convert full table scans into fast B-tree searches.

9. References & Primary Sources

1. **Mozilla Developer Network (MDN):** *Web Speech API Specification & SpeechRecognition Guide*, W3C Community Group.
2. **PostgreSQL Global Development Group:** *PostgreSQL 15 Documentation: Row Security Policies, Generated Columns & Explain Plans*, postgresql.org/docs.
3. **Supabase Inc.:** *Supabase Architecture, Row Level Security (RLS), and Postgres Audit Guides*, supabase.com/docs.
4. **Alpha Cephei:** *Vosk Speech Recognition Toolkit & WebAssembly Runtime*, Apache 2.0 License, alphacephei.com/vosk.
5. **Radford, A., et al.:** *Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)*, OpenAI, arXiv:2212.04356.
6. **Mao, Toby:** *SQLGlot: A Modular Python SQL Parser, Transpiler, Optimizer, and Engine*, github.com/tobymao/sqlglot.
7. **OWASP Foundation:** *SQL Injection Prevention Cheat Sheet & Defense in Depth Principles*, cheatsheetseries.owasp.org.